



A25 – Data Pipeline 1

XML-Based Sports Club Database Platform

Group Members

Ganesan Masimalai (DE)

Mohamed Ramalaawie Mohamed Illyas (DE)

Vishnuvardhan Babu Barri (DE)

Vidwan Reddy Nimma (DE)

Academic Year: 2025 – 2027

Contents

I. Introduction	2
II. XML Schema & Database Design	2
III. HTML Visualisation Developer	3
IV. Format Transformation	4
V. Developer & Technical Writer	5
VI. Results	6
VII. Conclusion	7

I. Introduction

This report documents the design and implementation of the A25 Data Pipeline 1 project — an XML-Based Sports Club Database Platform. The objective was to build a complete data pipeline encompassing XML schema design, data population, XSLT-based transformation into multiple output formats (HTML, XML, JSON), and a Python automation pipeline.

The project was completed by a team of four students, each responsible for a defined specialisation. This document details the contributions, tools, technical decisions, and results produced by each member, followed by a consolidated results summary and conclusion.

The platform models a sports club containing nine entity types — Members, Coaches, Facilities, Teams, TrainingSessions, Competitions, Equipment, Memberships, and Bookings — and demonstrates ten distinct XSLT transformation scenarios producing browser-ready HTML views and machine-readable XML/JSON exports.

Team Roles and Contributions

Name	Role	Key Deliverables	%	Member
Ganesan Masimalai	Schema & DB Designer	sportsclub.xsd, sportsclub.xml, validation	25%	Member 1
Mohamed Raihaan Illyas	HTML Visualisation Developer	Stylesheets 1–6, output_1.html – output_6.html	25%	Member 2
Vishnuvardhan Babu Barri	Format Transformation Specialist	Stylesheets 7–10, output 7–10, bookings_schema.json	25%	Member 3
Vidwan Reddy Nimma	Developer & Technical Writer	main.py, report.pdf	25%	Member 4

II. XML Schema & Database Design

Member 1 — Ganesan Masimalai (DE)

Overall Modeling Approach

The XML schema (sportsclub.xsd) models a sports club as a flat collection of nine top-level entity containers inside a single root element <SportsClub>: Members, Coaches, Facilities, Teams, TrainingSessions, Competitions, Equipment, Memberships, and Bookings. Each entity type is defined as a named complex Type with a strict xs:sequence, ensuring a predictable element order and enabling robust XPath navigation in XSLT stylesheets.

Cross-entity relationships are expressed through reference ID elements (e.g. CoachRef, FacilityRef, MemberRef) rather than nesting, keeping the schema modular and each entity independently queryable. Referential integrity is enforced at schema level via xs:key / xs:keyref declarations on the root element, functioning similarly to foreign-key constraints in a relational database.

Fine-Grained Type Definitions

Rather than relying on generic `xs:string` for all fields, the schema defines eight custom simple types to enforce strict domain-level validation at parse time:

- **AgeType:** `xs:integer` restricted to [5, 100] — prevents biologically implausible values.
- **EmailType:** `xs:string` with pattern `[^@]+@[^@]+\.[^@]+` — enforces basic e-mail format.
- **PhoneType:** `xs:string` with pattern `[0-9+ -]{7,20}` — accepts international formats.
- **GenderType:** enumeration {Male, Female, Other}.
- **MembershipCategoryType:** enumeration {Basic, Premium, VIP, Student}.
- **BookingStatusType:** enumeration {Confirmed, Pending, Cancelled}.
- **PriceType:** `xs:decimal` `>= 0` — prevents negative fees.
- **CapacityType:** `xs:integer` `>= 1` — a facility must hold at least one person.

Modeling Problem: Referential Integrity Without Nesting

A training session involves a coach and a facility. Two design options were evaluated: (a) nest coach and facility data directly inside each session element, or (b) store them separately and use ID references.

Option (a) — nesting — duplicates data: a coach appearing in ten sessions is stored ten times, making updates error-prone and inflating document size. Option (b) — reference IDs (chosen) — stores each entity once. Each `<TrainingSession>` holds a `<CoachRef>` and a `<FacilityRef>`; the XSD enforces correctness through `xs:keyref` constraints.

Advantages of Option (b): no data duplication; a coach update propagates automatically; XPath cross-references remain concise. The trade-off is that XSLT transformations must perform a join via `key()` or `generate-id()` to retrieve related data, adding moderate complexity to stylesheets.

III. HTML Visualisation Developer

Member 2 — Mohamed Ramalaawie Ilyas (DE)

Overview of Scenarios 1–6

Member 2 was responsible for developing six XSLT stylesheets (`stylesheet_1.xml` through `stylesheet_6.xml`) that transform the XML database into styled, browser-ready HTML pages. Each stylesheet targets a specific entity type and applies formatting rules to produce a readable, navigable report.

#	Stylesheet	Output	Description
1	stylesheet_1.xml	output_1.html	Member Directory — full member list with contact details
2	stylesheet_2.xml	output_2.html	Weekly Training Schedule — sessions sorted by date/time
3	stylesheet_3.xml	output_3.html	Team Overview — teams with rosters and coach information
4	stylesheet_4.xml	output_4.html	Facility Availability — facilities with capacity and status
5	stylesheet_5.xml	output_5.html	Competition Results — results grouped by team
6	stylesheet_6.xml	output_6.html	Membership Report — membership status and renewal dates

Technical Approach

Each stylesheet uses `xsl:for-each` and `xsl:sort` to iterate and order records, and `xsl:if` / `xsl:choose` for conditional formatting. HTML tables are generated with semantic markup (`<table>`, `<thead>`, `<tbody>`), and CSS inline styles are applied to produce colour-coded headers and alternating row fills. All six outputs were rendered and reviewed in Chrome and Firefox to confirm cross-browser compatibility.

Low-cardinality categorical features (protocol, traffic type, packet type, severity level, network segment, and action taken) are displayed in grouped summary tables within the HTML views, reducing visual noise while retaining completeness of the underlying data.

Working Environment

- **Saxon HE (Java):** Used as the XSLT 1.0 processor for offline testing of all six stylesheets before integration into the Python pipeline.
- **Chrome / Firefox:** Used for rendering and visual review of all HTML output files.
- **VS Code:** Primary editor for XSL authoring with XML syntax highlighting.

IV. Format Transformation

Member 3 — Vishnuvardhan Babu Barri (DE)

Overview of Scenarios 7–10

Member 3 developed four XSLT stylesheets (stylesheet_7.xsl through stylesheet_10.xsl) that export the XML data into structured XML and JSON formats suitable for downstream system integration and API consumption.

#	Stylesheet	Output	Description
7	stylesheet_7.xsl	output_7.xml	Member Contact List exported in a new XML format
8	stylesheet_8.xsl	output_8.xml	Training Session schedule exported as XML (ScheduleExport)
9	stylesheet_9.xsl	output_9.json	Bookings data exported as JSON array
10	stylesheet_10.xsl	output_10.json	Equipment inventory exported as JSON array

Deep Dive: Scenario 9 — Bookings to JSON

Stylesheet 9 converts the XML Bookings entity into a well-formed JSON array. This is technically non-trivial in XSLT 1.0 because the language provides no native JSON serialisation method; every character of the JSON structure must be manually constructed using `xsl:text`, `xsl:value-of`, and careful comma management.

- **Trailing comma problem:** JSON forbids a comma after the last array element. Since XSLT 1.0 has no lookahead, the stylesheet uses `position() != last()` inside an `xsl:if` to conditionally emit the comma only between elements.
- **Mixed content types:** `bookingID`, `memberID`, `facilityID`, `date`, and `time` fields are JSON strings (quoted), while numeric fields must remain unquoted. The stylesheet explicitly wraps string values in double quotes via `xsl:text`.
- **Output method:** XSLT 1.0 does not support `method='json'`; accordingly, `xsl:output method='text'` is used, and every character — braces, brackets, colons, newlines — is hard-coded as a text node.
- **Validation:** The output is validated against `bookings_schema.json` (JSON Schema Draft-07), which enforces ID patterns (e.g. `B[0-9]{3}`), ISO date format, and the `BookingStatus` enumeration.

Tools Used

- **Saxon HE (Java):** XSLT 1.0 processor for offline stylesheet testing.
- **jsonschema.dev:** Online JSON Schema validator used to verify `output_9.json` and `output_10.json` against `bookings_schema.json`.

V. Developer & Technical Writer

Member 4 — Vidwan Reddy Nimma (DE)

Python Pipeline (main.py)

Member 4 developed `main.py`, the Python automation pipeline that orchestrates all four transformation steps using the `lxml` library, ensuring reproducible and automated end-to-end processing of the XML source data.

- **Step 1 — Parse:** `etree.parse()` loads `sportsclub.xml` into an `ElementTree`, checking well-formedness before any further processing.
- **Step 2 — Load Schema:** `etree.XMLSchema()` compiles `sportsclub.xsd` into a reusable validator object, avoiding repeated disk reads.
- **Step 3 — Validate:** `schema.validate(xml_tree)` executes XSD validation; all constraint violations are printed with their line numbers via `schema.error_log`, enabling precise debugging.
- **Step 4 — Transform:** For each stylesheet, `etree.XSLT()` compiles the XSL and applies it to the XML tree. XML outputs are serialised with `etree.tostring(pretty_print=True)`; JSON/text outputs use `str(result)` to preserve raw character sequences.

VI. Results

Pipeline Execution Results

The Python pipeline (`main.py`) executed successfully on the first complete run against the validated XML database. All four XSLT stylesheets (7–10) were compiled and applied without errors, producing the expected output files in the correct formats.

Step	Operation	Input	Output	Status
1	Parse XML	sportsclub.xml	ElementTree object	OK — Well-formed
2	Load XSD Schema	sportsclub.xsd	XMLSchema object	OK — Schema loaded
3	Validate XML	ElementTree + Schema	Validation report	OK — XML is VALID
4a	Apply stylesheet_7.xsl	sportsclub.xml	output_7.xml	OK — Written (XML)
4b	Apply stylesheet_8.xsl	sportsclub.xml	output_8.xml	OK — Written (XML)
4c	Apply stylesheet_9.xsl	sportsclub.xml	output_9.json	OK — Written (text)
4d	Apply stylesheet_10.xsl	sportsclub.xml	output_10.json	OK — Written (text)

HTML Output Results (Scenarios 1–6)

All six HTML stylesheets produced valid, browser-renderable output files. Each file was reviewed in Chrome and Firefox with no rendering discrepancies observed. The outputs correctly reflect the underlying XML data, including cross-referenced entities resolved via XSLT `key()` lookups.

JSON Validation Results

The JSON outputs (output_9.json and output_10.json) were validated against their respective JSON Schema Draft-07 specifications using jsonschema.dev. Both files passed validation without errors, confirming correct adherence to ID patterns, enumeration constraints, and data type requirements.

XML Export Validation

The XML export files (output_7.xml and output_8.xml) were validated using xmllvalidation.com against the expected export structure. Both files were confirmed well-formed and schema-compliant, with all cross-referenced entities correctly resolved.

VII. Conclusion

This project successfully demonstrates the implementation of an XML-based sports club management platform using XML technologies. The project combines XML Schema validation, XSLT transformations, JSON generation, and Python-based XML processing to create a structured and functional data management system.

The modular XML design improves maintainability and scalability while the XSLT stylesheets provide meaningful visualizations and data exchange formats.

Through this project, the team gained practical experience in:

- XML database modeling
- Schema validation
- XSLT programming
- XPath querying
- Python XML APIs
- Data transformation techniques

The project shows how XML technologies can effectively support real-world sports club management applications.

VIII. AI Assistance Declaration

AI tools such as ChatGPT were used during some stages of the project as supporting tools.

Areas Where AI Was Used

- Understanding XML Schema syntax

- Improving XPath and XSLT logic
- Structuring the project report
- Debugging Python validation errors
- Generating documentation ideas

Example Prompts Used

- “Generate an XSLT stylesheet to display sports club members in an HTML table.”
- “Explain how to validate XML against XSD using Python lxml.”
- “Provide an XML Schema example using KEY and KEYREF constraints.”

Modifications Performed by the Team

The output generated were manually reviewed and modified:

- XPath expressions were optimized
- HTML formatting was improved
- Schema constraints were adjusted
- Python error handling was extended

AI assistance was used only for guidance purposes while the final implementation, testing, and validation were completed by the project team.